ASSIGNMENT-12.3

Roll no:2303A52074
Batch-37

Task 1: Sorting Student Records for Placement Drive

Prompt

#Create a Python program to store student records (Name, Roll No, CGPA).
#Sort them in descending order using Quick Sort and Merge Sort.
#Compare their runtime performance.
#Display the top 10 students by giving rankings based on their CGPA.

Code

ai 12.3.py > merge_sort

```python
#Create a Python program to store student records (Name, Roll No, CGPA).
#Sort them in descending order using Quick Sort and Merge Sort.
#Compare their runtime performance.
#Display the top 10 students by giving rankings based on their CGPA.
import time
class Student:
    def __init__(self, name, roll_no, cgpa):
        self.name = name
        self.roll_no = roll_no
        self.cgpa = cgpa
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2].cgpa
    left = [x for x in arr if x.cgpa > pivot]
    middle = [x for x in arr if x.cgpa == pivot]
    right = [x for x in arr if x.cgpa < pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    result = []
```

```python
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].cgpa > right[j].cgpa:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
students = [
    Student("Alice", 1, 3.5),
    Student("Bob", 2, 3.8),
    Student("Charlie", 3, 3.2),
    Student("David", 4, 3.9),
    Student("Eve", 5, 3.6),
    Student("Frank", 6, 3.7),
    Student("Grace", 7, 3.4),
    Student("Heidi", 8, 3.1),
    Student("Ivan", 9, 3.0),
    Student("Judy", 10, 3.3)
]
# Measure runtime for Quick Sort
```


```python
start_time = time.time()
```


```python
# Measure runtime for Quick Sort
start_time = time.time()
sorted_students_quick = quick_sort(students)
end_time = time.time()
print("Quick Sort Runtime: {:.6f} seconds".format(end_time - start_time))
# Measure runtime for Merge Sort
start_time = time.time()
sorted_students_merge = merge_sort(students)
end_time = time.time()
print("Merge Sort Runtime: {:.6f} seconds".format(end_time - start_time))
# Display top 10 students
print("\nTop 10 Students:")
for i, student in enumerate(sorted_students_quick[:10], start=1):
    print(f"Rank {i}: {student.name}, Roll No: {student.roll_no}, CGPA: {student.cgpa}")
```

Output

```
Quick Sort Runtime: 0.000018 seconds
Merge Sort Runtime: 0.000037 seconds

Top 10 Students:
Rank 1: David, Roll No: 4, CGPA: 3.9
Rank 2: Bob, Roll No: 2, CGPA: 3.8
Rank 3: Frank, Roll No: 6, CGPA: 3.7
Rank 4: Eve, Roll No: 5, CGPA: 3.6
Rank 5: Alice, Roll No: 1, CGPA: 3.5
Rank 6: Grace, Roll No: 7, CGPA: 3.4
Rank 7: Judy, Roll No: 10, CGPA: 3.3
Rank 8: Charlie, Roll No: 3, CGPA: 3.2
Rank 9: Heidi, Roll No: 8, CGPA: 3.1
Rank 10: Ivan, Roll No: 9, CGPA: 3.0
```

Explanation

This program stores student records using a Student class with name, roll number, and CGPA.
It sorts the students in descending order of CGPA using Quick Sort and Merge Sort.
The runtime of both algorithms is measured using the time module.
Finally, it displays the top 10 students with rankings based on their CGPA.

Task 2: Implementing Bubble Sort with AI Comments

Prompt

Implement Bubble Sort in Python with inline comments explaining the logic and provide its time complexity analysis.

Code

```python
#Implement Bubble Sort in Python with inline comments explaining the logic and provide its t
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all elements in the array
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False
        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Compare adjacent elements and swap if they are in the wrong order
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]  # Swap
                swapped = True
        # If no swapping occurred, the array is already sorted
        if not swapped:
            break
# Time Complexity Analysis:
# The time complexity of Bubble Sort is O(n^2) in the worst and average cases because it rec
# In the best case, when the array is already sorted, the time complexity is O(n) because it
# the array is sorted without performing any swaps.
# Example usage
arr = [64, 34, 25, 12, 22, 11, 90]
bubble_sort(arr)
print("Sorted array is:", arr)
```

Output

```
Sorted array is: [11, 12, 22, 25, 34, 64, 90]
```

Explanation

In this program, I implemented the Bubble Sort algorithm to sort the array in ascending order using nested loops and swapping adjacent elements. I also added a swapped flag to stop the algorithm early if the array becomes sorted before completing all passes. I included inline comments explaining the logic and provided time complexity analysis.

Task 3: Quick Sort and Merge Sort Comparison

Promp

Complete recursive Quick Sort and Merge Sort functions in Python, add docstrings, compare them on different types of lists, and explain their time complexities.

Code

```python
#Complete recursive Quick Sort and Merge Sort functions in Python, add docstrings, compare them on different ty
def quick_sort(arr):
    """
    Sorts an array in descending order using the Quick Sort algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: A new list sorted in descending order.
    """
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x > pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x < pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr):
    """
    Sorts an array in descending order using the Merge Sort algorithm.
    Parameters:
    arr (list): The list of elements to be sorted.
    Returns:
    list: A new list sorted in descending order.
    """
```

```python
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    """
    Merges two sorted lists into a single sorted list in descending order.
    Parameters:
    left (list): The first sorted list.
    right (list): The second sorted list.
    Returns:
    list: A merged list sorted in descending order.
    """
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] > right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
```

```python
def merge(left, right):
    result.extend(right[j:])
    return result
# Example usage and performance comparison
import time
# Test on different types of lists
test_lists = {
    "Random List": [64, 34, 25, 12, 22, 11, 90],
    "Sorted List": [1, 2, 3, 4, 5, 6, 7],
    "Reverse Sorted List": [7, 6, 5, 4, 3, 2, 1]
}
for name, lst in test_lists.items():
    # Measure runtime for Quick Sort
    start_time = time.time()
    sorted_quick = quick_sort(lst)
    end_time = time.time()
    print(f"{name} - Quick Sort Runtime: {end_time - start_time:.6f} seconds")

    # Measure runtime for Merge Sort
    start_time = time.time()
    sorted_merge = merge_sort(lst)
    end_time = time.time()
    print(f"{name} - Merge Sort Runtime: {end_time - start_time:.6f} seconds")

    print(f"{name} - Sorted with Quick Sort: {sorted_quick}")
    print(f"{name} - Sorted with Merge Sort: {sorted_merge}\n")
# Time Complexity Analysis:
# Quick Sort has an average and worst-case time complexity of O(n log n) when the pivot selection is good,
# but it can degrade to O(n^2) if the pivot selection is poor (e.g., when the smallest or largest element is a
# Merge Sort has a consistent time complexity of O(n log n)
# in all cases because it always divides the array into two halves and processes each half independently.
```

Output

```
Random List - Quick Sort Runtime: 0.000018 seconds
Random List - Merge Sort Runtime: 0.000039 seconds
Random List - Sorted with Quick Sort: [90, 64, 34, 25, 22, 12, 11]
Random List - Sorted with Merge Sort: [90, 64, 34, 25, 22, 12, 11]

Sorted List - Quick Sort Runtime: 0.000017 seconds
Sorted List - Merge Sort Runtime: 0.000034 seconds
Sorted List - Sorted with Quick Sort: [7, 6, 5, 4, 3, 2, 1]
Sorted List - Sorted with Merge Sort: [7, 6, 5, 4, 3, 2, 1]

Reverse Sorted List - Quick Sort Runtime: 0.000027 seconds
Reverse Sorted List - Merge Sort Runtime: 0.000041 seconds
Reverse Sorted List - Sorted with Quick Sort: [7, 6, 5, 4, 3, 2, 1]
Reverse Sorted List - Sorted with Merge Sort: [7, 6, 5, 4, 3, 2, 1]
```

Explanation

In this program, I implemented Quick Sort and Merge Sort using recursion to sort lists in descending order. I added proper docstrings to explain the purpose, parameters, and return values of each function. I compared both algorithms by testing them on random, sorted, and reverse-sorted lists and measured their runtime using the time module. I also analyzed their time complexities and understood that Quick Sort can become $O(n^2)$ in the worst case, while Merge Sort consistently runs in $O(n \log n)$ in all cases.

Task 4 (Real-Time Application – Inventory Management System)

Prompt

#Build a Python inventory system with Product ID, Name, Price, and Quantity.
#Suggest the best search and sort algorithms, justify your choice,
#and give them with a table showing Operation ,Algorithm ,Reason

Code

```python
#Build a Python inventory system with Product ID, Name, Price, and Quantity.
#Suggest the best search and sort algorithms, justify your choice,
#and give them with a table showing Operation ,Algorithm ,Reason
class Product:
    def __init__(self, product_id, name, price, quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.quantity = quantity
class Inventory:
    def __init__(self):
        self.products = []
    def add_product(self, product):
        self.products.append(product)
    def search_product(self, product_id):
        # Using Binary Search for efficient searching
        self.products.sort(key=lambda x: x.product_id)  # Ensure products are sorted by ID
        left, right = 0, len(self.products) - 1
        while left <= right:
            mid = (left + right) // 2
            if self.products[mid].product_id == product_id:
                return self.products[mid]
            elif self.products[mid].product_id < product_id:
                left = mid + 1
            else:
                right = mid - 1
        return None
    def sort_products_by_price(self):
        # Using Merge Sort for efficient sorting of products by price
        self.products = merge_sort(self.products)
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].price < right[j].price:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

```python
# Example usage
inventory = Inventory()
inventory.add_product(Product(1, "Laptop", 999.99, 10))
inventory.add_product(Product(2, "Smartphone", 499.99, 20))
inventory.add_product(Product(3, "Headphones", 199.99, 15))
# Search for a product
product = inventory.search_product(2)
if product:
    print(f"Product found: {product.name}, Price: {product.price}, Quantity: {product.quantity}")
else:
    print("Product not found.")
# Sort products by price
inventory.sort_products_by_price()
print("Products sorted by price:")
for product in inventory.products:
    print(f"{product.name}: ${product.price}")
# Operation → Algorithm → Reason
# Search → Binary Search → Efficient for sorted lists, O(log n) time complexity
# Sort → Merge Sort → Stable and efficient for large datasets, O(n log n) time complexity
```

Output

```
Product found: Smartphone, Price: 499.99, Quantity: 20
Products sorted by price:
Headphones: $199.99
Smartphone: $499.99
Laptop: $999.99
```

Explanation

In this program, I created a Product class to store product ID, name, price, and quantity. I then created an Inventory class to manage products by adding, searching, and sorting them. For searching, I used Binary Search because it is efficient for large sorted datasets with O(log n) time complexity. For sorting by price, I implemented Merge Sort since it is stable and has O(n log n) time complexity, making it suitable for large inventories.

Task 5: Real-Time Stock Data Sorting & Searching

Prompt

#Build a Python stock analysis tool that sorts stocks by percentage change using Heap Sort
and searches by symbol using a Hash Map. Compare performance with built-in functions and explain trade-offs.

Code

```python
#Build a Python stock analysis tool that sorts stocks by percentage change using Heap Sort
# and searches by symbol using a Hash Map. Compare performance with built-in functions and explain trade-offs.
import heapq
class Stock:
    def __init__(self, symbol, price, percentage_change):
        self.symbol = symbol
        self.price = price
        self.percentage_change = percentage_change
class StockAnalysisTool:
    def __init__(self):
        self.stocks = []
        self.stock_map = {}
    def add_stock(self, stock):
        self.stocks.append(stock)
        self.stock_map[stock.symbol] = stock
    def sort_stocks_by_percentage_change(self):
        # Using Heap Sort to sort stocks by percentage change
        return heapq.nlargest(len(self.stocks), self.stocks, key=lambda x: x.percentage_change)
    def search_stock_by_symbol(self, symbol):
        # Using Hash Map for efficient searching by symbol
        return self.stock_map.get(symbol, None)
# Example usage
stock_tool = StockAnalysisTool()
stock_tool.add_stock(Stock("AAPL", 150.00, 2.5))
stock_tool.add_stock(Stock("GOOGL", 2800.00, -1.2))
stock_tool.add_stock(Stock("AMZN", 3400.00, 3.0))
```

```python
stock_tool.add_stock(Stock("AMZN", 3400.00, 3.0))
# Sort stocks by percentage change
sorted_stocks = stock_tool.sort_stocks_by_percentage_change()
print("Stocks sorted by percentage change:")
for stock in sorted_stocks:
    print(f"{stock.symbol}: {stock.percentage_change}%")
# Search for a stock by symbol
stock = stock_tool.search_stock_by_symbol("GOOGL")
if stock:
    print(f"Stock found: {stock.symbol}, Price: {stock.price}, Percentage Change: {stock.percentage_change}%")
else:
    print("Stock not found.")
# Performance Comparison
# Heap Sort has a time complexity of O(n log n) for sorting, which is efficient for large datasets.
# The built-in sorted() function in Python uses Timsort,
# which has a time complexity of O(n log n) in the worst case and O(n) in the best case (when the list is alread
# Hash Map provides O(1) average time complexity for search operations, while a linear search would have O(n) ti
# Trade-offs:
# Heap Sort is efficient for sorting large datasets but may not be as fast as Timsort for small or partially sor
# Hash Map allows for fast lookups but can consume more memory compared to a list, especially if there are many
```

Output

```
Stocks sorted by percentage change:
AMZN: 3.0%
AAPL: 2.5%
GOOGL: -1.2%
Stock found: GOOGL, Price: 2800.0, Percentage Change: -1.2%
```

Explanation

Created a stock analysis tool where added stock details like symbol, price, and percentage change. When sorted the stocks, the program used Heap Sort to arrange them based on percentage gain or loss. When searched for a stock symbol, the program used a dictionary (Hash Map) to quickly find and return the stock details. Finally, compared the performance and understood that sorting takes O(n log n) time and searching using a Hash Map takes O(1) time on average.